POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

A lightweight coefficient extractor in C

WILLIAM ZHANG
Département de Electrical Engineering

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Electrical Engineering

July 2026

© William Zhang, 2026.

POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

Ce mémoire intitulé :

A lightweight coefficient extractor in C

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Danielle DUBOIS, présidente
Sébastien VIGOT, membre et directeur de recherche
Marie COUTURE, membre et codirectrice de recherche
Jean TREMBLAY, membre
Joseph BROWN, membre externe

DEDICATION

À tous mes amis du labos,
vous me manquerez...

ACKNOWLEDGEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE OF CONTENTS

DEDICATION iii

ACKNOWLEDGEMENTS iv

RÉSUMÉ v

ABSTRACT vi

LIST OF TABLES ix

LIST OF FIGURES x

LIST OF SYMBOLS AND ACRONYMS xi

LIST OF APPENDICES xii

CHAPTER 1 INTRODUCTION 1
- 1.1 Definitions and basic concepts 1
- 1.2 Problem statement 1
- 1.3 Research objectives 1
- 1.4 Thesis outline 1

CHAPTER 2 LITERATURE REVIEW 2
- 2.1 Historical Background 2
- 2.2 Related work 3
- 2.3 Summary 3

CHAPTER 3 Setting the stage 4

CHAPTER 4 Handling parameters 10

CHAPTER 5 Benchmark results 11

CHAPTER 6 CONCLUSION 12
- 6.1 Synthèse des travaux / Summary of Works 12
- 6.2 Limitations de la solution proposée / Limitations 12
- 6.3 Améliorations futures / Future Research 12

REFERENCES 13

APPENDICES 15

LIST OF TABLES

LIST OF FIGURES

Figure 3.1 Expression tree for the affine expression `sum(A @ x - b) + C @ y`. . 5

Figure 3.2 Coefficient matrix accumulation for `sum(A @ x)`. Each node composes its operator with the child's coefficient matrix via sparse matrix multiplication. 8

Figure 3.3 CSR representation before and after applying `sum`. The 3-row matrix collapses to 1 row by setting `indptr` $= [0, 6]$. Duplicate column indices (0 and 2) represent values that sum together. The `indices` and `data` arrays remain unchanged—no matrix multiplication required. 9

LIST OF SYMBOLS AND ACRONYMS

IETF	Internet Engineering Task Force
OSI	Open Systems Interconnection

LIST OF APPENDICES

Appendix A Démo 15
Appendix B Encore une annexe / Another Appendix 16
Appendix C Une dernière annexe / The Last Appendix 17

CHAPTER 1 INTRODUCTION

1.1 Definitions and basic concepts

1.2 Problem statement

1.3 Research objectives

1.4 Thesis outline

CHAPTER 2 LITERATURE REVIEW

2.1 Historical Background

Solving an optimization problem on a computer requires translating the mathematical formulation into a data format that solvers can process. For linear programs, this means constructing the constraint matrix A, the right-hand side vector b, and the objective coefficients c that define the *standard form*,

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned} \tag{2.1}$$

Manual encoding In the early days of linear programming, analysts manually computed and encoded each coefficient onto punch cards. This process was tedious, error-prone and became impractical as linear programming found more and more applications in large-scale industrial problems [1].

MPS format IBM's Mathematical Programming System (MPS) file format, developed in the 1960s, established a standardized file format for representing linear programs [2]. The MPS format uses fixed-width fields designed for 80-column punch cards, with sections for rows, columns, and coefficients.

Matrix generators To automate the creation of MPS files, *matrix generators* emerged in the late 1960s; they are procedural programs that compute the problem coefficients through explicit loops and output the results in MPS format [3]. Commercial systems such as MaGen [4], GAMMA [5], and DATAFORM [6] provided libraries and tools to facilitate matrix generation for various application domains.

A modern approach However, modelers naturally think in terms of sets, indices, and algebraic expressions, not in terms of individual matrix entries. Matrix generators simplified the encoding process but still required modelers to translate their mathematical formulations into procedural code that explicitly constructs the coefficient matrices. This suggests that the gap between the modeler's representation of the problem and the solver's required data format hasn't been resolved yet [1].

Algebraic modeling languages In the late 1970s, *algebraic modeling languages* emerged as formal languages designed to accept problem descriptions that closely mirror standard mathematical notation [7]. By working directly with the modeler's representation, these languages simplify verification (the model can be inspected as written), modification (changing the problem does not require rewriting procedural code), and communication (the formulation serves as its own documentation). In the late 1980s and early 1990s, commercial algebraic modeling systems such as GAMS [8], AIMMS [9] and AMPL [10] became widely adopted in both academia and industry (and they are still widely used today).

2.2 Related work

In this section we review the high level implementation of three widely used open-source modeling languages: JuMP [11, 12], Pyomo [13, 14] and CVXPY.

JuMP JuMP is a domain-specific modeling language for mathematical optimization embedded in Julia. It is based on the `MathOptInterface` (MOI) [15] abstraction layer.

Pyomo Pyomo is a Python-based open-source optimization modeling language.

CVXPY CVXPY is a Python-embedded modeling language for convex optimization problems.

2.3 Summary

CHAPTER 3 Setting the stage

Functional convex programming Users specify problems—by providing expression trees for each function—as the standard "functional" form for convex optimization

$$\begin{array}{ll} \text{minimize} & f_0(x;\theta) \\ \text{subject to} & f_i(x;\theta) \leq 0,\ i = 1,\ldots,m_1, \\ & g_i(x;\theta) = 0,\ i = 1,\ldots,m_2, \end{array} \tag{3.1}$$

with variable $x \in \mathbb{R}^n$ and parameter vector $\theta \in \mathbb{R}^p$ [16]. When θ is fixed, (3.1) is a problem instance, otherwise it defines a problem family.

Quadratic conic programming CVXPY transforms the problem in (3.1) into an equivalent quadratic conic program (QCP) of the form

$$\begin{array}{ll} \text{minimize} & \frac{1}{2}x^T P x + q^T x \\ \text{subject to} & b - Ax \in \mathcal{K}, \end{array} \tag{3.2}$$

where $x \in \mathbf{R}^n$ is the variable, $\mathcal{K} \subseteq \mathbf{R}^m$ is a closed convex cone, and the *problem data* are $P \in \mathbf{S}^n_+$, $q \in \mathbf{R}^n$, $A \in \mathbf{R}^{m\times n}$, and $b \in \mathbf{R}^m$. CVXPY supports problems where the cone is a product of the zero cone ($\{0\}$), the nonnegative cone ($\mathbf{R}_+$), the second-order cone ($\mathcal{K}_{\text{soc}}$), the power cone ($\mathcal{K}_{\text{pow}}$), the exponential cone ($\mathcal{K}_{\text{exp}}$), and the cone of positive-semi-definite matrices ($\mathbf{S}_+$).

Linear operator tree CVXPY's canonicalization guarantees that all constraints and the objective function in (3.1) are affine functions of the variable x. An affine function can be expressed as the composition of linear functions and addition of constants. CVXPY represents each affine function as an *expression tree*, where each node is a linear function or *atom*.

A first example When you specify a single variable in CVXPY, such as

```
x = cp.Variable(4)
```

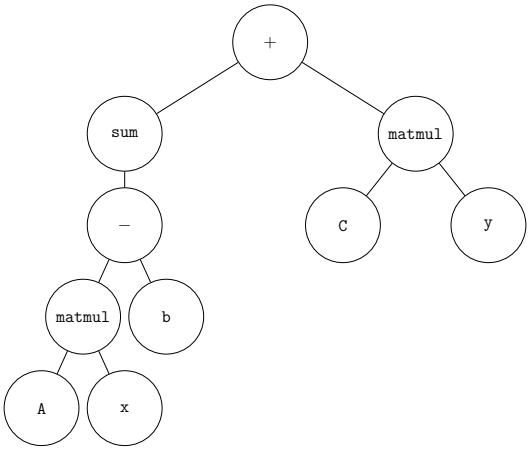


Figure 3.1 Expression tree for the affine expression `sum(A @ x - b) + C @ y`.

the internal representation is the identity linear operator of size 4×4:

$$\begin{array}{c} \begin{array}{cccc} x_1 & x_2 & x_3 & x_4 \end{array} \\ \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{array}$$

A second example When you specify a sum operator on a variable in CVXPY, such as

```
x = cp.Variable(4)
y = cp.sum(x)
```

the internal representation is a 1×4 matrix of all ones:

$$\begin{array}{c} \begin{array}{cccc} x_1 & x_2 & x_3 & x_4 \end{array} \\ \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \end{array}$$

Applying this operator to the variable vector yields the sum:

$$\begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = x_1 + x_2 + x_3 + x_4$$

A matrix example When you specify a matrix variable in CVXPY, such as

```
X = cp.Variable((3, 2))
```

the entries of the matrix are vectorized in column-major order, and the internal representation is the identity linear operator of size 6×6:

$$\begin{array}{c} \begin{array}{cccccc} X_{11} & X_{21} & X_{31} & X_{12} & X_{22} & X_{32} \end{array} \\ \left[\begin{array}{cccccc} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{array}\right] \end{array}$$

A composition example When you specify a more complex linear function in CVXPY, such as

```
x = cp.Variable(3)
A = np.random.randn(5, 3)
b = np.random.randn(5)
y = cp.sum(A @ x + b)
```

the internal representation is the composition of three linear operators:

- the matrix multiplication operator $A \in \mathbf{R}^{5\times 3}$,
- the vector addition operator $+b$ where $b \in \mathbf{R}^5$,
- the sum operator $\mathbf{1}^T \in \mathbf{R}^{1\times 5}$.

The constant b is incorporated by augmenting the matrix with a column for the constant term. The resulting linear operator is $\mathbf{1}^T[A \mid b] \in \mathbf{R}^{1\times 4}$:

$$\underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}}_{\mathbf{1}^T} \underbrace{\left[\begin{array}{ccc|c} A_{11} & A_{12} & A_{13} & b_1 \\ A_{21} & A_{22} & A_{23} & b_2 \\ A_{31} & A_{32} & A_{33} & b_3 \\ A_{41} & A_{42} & A_{43} & b_4 \\ A_{51} & A_{52} & A_{53} & b_5 \end{array}\right]}_{[A|b]} = \begin{array}{cccc} x_1 & x_2 & x_3 & 1 \end{array} \\ \left[\begin{array}{ccc|c} \sum_i A_{i1} & \sum_i A_{i2} & \sum_i A_{i3} & \sum_i b_i \end{array}\right]$$

A multi-variable example When you specify multiple variables and constraints in CVXPY, such as

```
x = cp.Variable()
y = cp.Variable(3)
z = cp.Variable(2)
constraints = [8*x == 28, cp.sum(y) + 5*cp.sum(z) == 58, z == 26]
```

the variables are concatenated into a single vector $[x, y_1, y_2, y_3, z_1, z_2]^T$. Each constraint is converted to the form $Ax + b = 0$, producing a linear operator with a constant column:

- `8*x == 28` becomes $8x - 28 = 0$:

$$\begin{array}{cccccc|c} x & y_1 & y_2 & y_3 & z_1 & z_2 & 1 \\ \hline 8 & 0 & 0 & 0 & 0 & 0 & -28 \end{array}$$

- `cp.sum(y) + 5*cp.sum(z) == 58` becomes $\sum y_i + 5\sum z_i - 58 = 0$:

$$\begin{array}{cccccc|c} x & y_1 & y_2 & y_3 & z_1 & z_2 & 1 \\ \hline 0 & 1 & 1 & 1 & 5 & 5 & -58 \end{array}$$

- `z == 26` becomes $z_1 - 26 = 0$ and $z_2 - 26 = 0$:

$$\begin{array}{cccccc|c} x & y_1 & y_2 & y_3 & z_1 & z_2 & 1 \\ \hline 0 & 0 & 0 & 0 & 1 & 0 & -26 \\ 0 & 0 & 0 & 0 & 0 & 1 & -26 \end{array}$$

These are then stacked into a single linear operator:

$$\begin{array}{cccccc|c} x & y_1 & y_2 & y_3 & z_1 & z_2 & 1 \\ \hline 8 & 0 & 0 & 0 & 0 & 0 & -28 \\ 0 & 1 & 1 & 1 & 5 & 5 & -58 \\ 0 & 0 & 0 & 0 & 1 & 0 & -26 \\ 0 & 0 & 0 & 0 & 0 & 1 & -26 \end{array}$$

The current approach: sparse matrix multiplication CVXPY's current approach is to convert the linear operator tree into a single sparse matrix. The tree is traversed in

post-order: each leaf node returns its coefficient matrix, and each internal node composes its children's matrices via sparse matrix multiplication. For example, in the expression `sum(A @ x)`, the traversal first returns A at the `matmul` node, then multiplies by $\mathbf{1}^T$ at the `sum` node, yielding $\mathbf{1}^T A$. The final matrix at the root is returned.

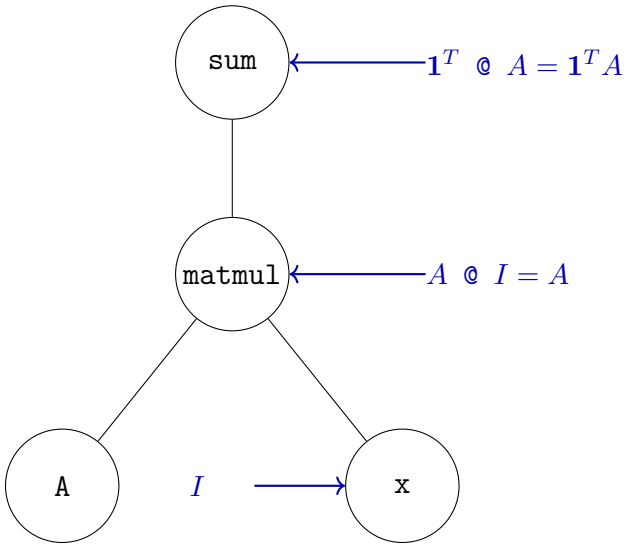


Figure 3.2 Coefficient matrix accumulation for `sum(A @ x)`. Each node composes its operator with the child's coefficient matrix via sparse matrix multiplication.

Limitations of current approach This approach requires storing intermediate sparse matrices at each node during traversal. For deep expression trees or large problem dimensions, this leads to significant memory overhead. Additionally, sparse matrix multiplication is computationally expensive and may produce matrices with more nonzeros than the operands, further increasing memory and runtime costs.

Our approach: direct encoding of linear operators We keep the coefficient matrix in CSR (Compressed Sparse Row) format and directly manipulate its index arrays without forming intermediate matrices. Each linear operator is implemented as a transformation on the CSR arrays `(indptr, indices, data)`. For example, the `sum` operator collapses all rows into row 0 by setting all entries of `indptr` to $[0, \texttt{nnz}]$, effectively mapping every row index to 0 without any matrix multiplication.

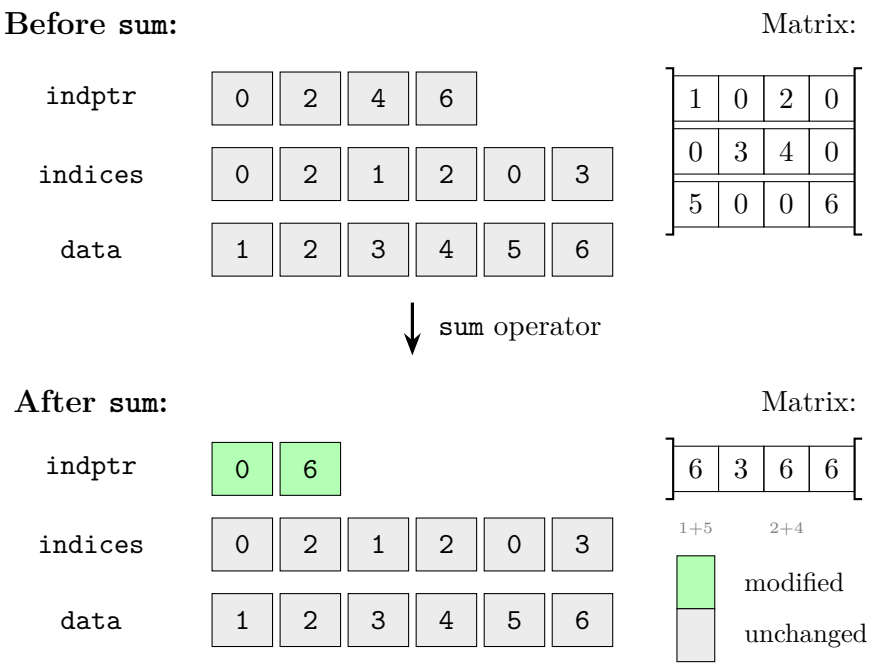


Figure 3.3 CSR representation before and after applying `sum`. The 3-row matrix collapses to 1 row by setting `indptr` $= [0, 6]$. Duplicate column indices (0 and 2) represent values that sum together. The `indices` and `data` arrays remain unchanged—no matrix multiplication required.

CHAPTER 4 Handling parameters

CHAPTER 5 Benchmark results

CHAPTER 6 CONCLUSION

Texte / Text.

6.1 Synthèse des travaux / Summary of Works

Texte / Text.

6.2 Limitations de la solution proposée / Limitations

6.3 Améliorations futures / Future Research

Texte / Text.

REFERENCES

[1] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," 1978.

[2] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[3] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[4] *MaGen Reference Manual*, Haverly Systems, Inc., Denville, NJ, 1977.

[5] *GAMMA 3.4 Programmer Reference*, Sperry Univac Computer Systems, St. Paul, MN, 1977.

[6] *MIPS III DATAFORM User Manual*, Ketron, Inc., Arlington, VA, 1975.

[7] R. Fourer, "Modeling languages versus matrix generators for linear programming," *ACM Trans. Math. Softw.*, vol. 9, no. 2, pp. 143–183, Jun. 1983. [Online]. Available: https://doi.org/10.1145/357456.357457

[8] A. Brooke, D. Kendrick, and A. Meeraus, *GAMS: A User's Guide.* Redwood City, CA: The Scientific Press, 1988.

[9] J. Bisschop, *AIMMS optimization modeling*, 2006.

[10] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[11] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[12] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[13] W. E. Hart, J.-P. Watson, and D. L. Woodruff, "Pyomo: modeling and solving mathematical programs in Python," *Mathematical Programming Computation*, vol. 3, no. 3, pp. 219–260, 2011.

[14] M. L. Bynum, G. A. Hackebeil, W. E. Hart, C. D. Laird, B. L. Nicholson, J. D. Siirola, J.-P. Watson, and D. L. Woodruff, *Pyomo–Optimization Modeling in Python*, 3rd ed. Springer Science & Business Media, 2021, vol. 67.

[15] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, "Mathoptinterface: a data structure for mathematical optimization problems," 2020. [Online]. Available: https://arxiv.org/abs/2002.03447

[16] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

APPENDIX A DÉMO

Texte de l'annexe A. Remarquez que la phrase précédente se termine par une lettre majuscule suivie d'un point. On indique explicitement cette situation à LaTeX afin que ce dernier ajuste correctement l'espacement entre le point final de la phrase et le début de la phrase suivante.

APPENDIX B ENCORE UNE ANNEXE / ANOTHER APPENDIX

Texte de l'annexe B en mode «landscape».

APPENDIX C UNE DERNIÈRE ANNEXE / THE LAST APPENDIX

Texte de l'annexe C.